

Code Explanation

SFDC Account Integration ETL Module Explanation

This is a Python script that implements an ETL (Extract, Transform, Load) pipeline for integrating Salesforce Account data from an Oracle source table to a target Oracle table using Apache Spark.

The script is designed to be executed as a standalone application, and it uses command-line arguments to pass the update date parameter.

Overview of the Code

The code is structured around several key functions that perform the ETL operations. It starts by configuring logging and defining Oracle connection configurations for the source and target tables. The main ETL process is orchestrated by the `extract\_transform\_load` function, which calls the `extract\_data`, `transform\_data`, and `load\_data` functions in sequence.

Step 1: Configuring Logging and Oracle Connections

The script begins by configuring logging using the Python `logging` module. It sets the log level to `INFO` and defines a log format that includes the timestamp, log level, and log message.

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)
logger = logging.getLogger( name )
```

The Oracle connection configurations for the source and target tables are defined as dictionaries (`ORACLE\_SOURCE\_CONFIG` and `ORACLE\_TARGET\_CONFIG`). These configurations include the JDBC URL, database table name, username, password, and driver.

```
ORACLE_SOURCE_CONFIG = {
    "url": "jdbc:oracle:thin:@UTDW014:1521:ORCL",
    "dbtable": "SFDC_ABS.DIM_MPE_SF_ACCOUNT",
    "user": "oracle_reader",
    "password": "{{secrets/oracle_reader}}",
    "driver": "oracle.jdbc.driver.OracleDriver",
    "fetchsize": "10000"
}
```

Step 2: Defining the Source Schema

The `get\_source\_schema` function defines the schema for the source Oracle table (`DIM\_MPE\_SF\_ACCOUNT`). The schema is defined using the `StructType` and `StructField` classes from the PySpark `sql.types` module.

```
def get_source_schema() -> StructType:
    return StructType([
        StructField("SK_ACCOUNT_ID", DecimalType(15, 0), False),
        StructField("DW_SF_ACCOUNT_ID", DecimalType(15, 0), False),
        # ... Additional fields would be defined here
```

1)

Step 3: Extracting Data from the Source Oracle Table

The `extract\_data` function extracts data from the source Oracle table based on the provided update date parameter. It constructs a SQL query with filters and uses the PySpark `read` API to read data from the Oracle table.

```
def extract_data(spark: SparkSession, update_dt: str) -> DataFrame:
    query = f"""
    SELECT *
    FROM SFDC_ABS.DIM_MPE_SF_ACCOUNT
    WHERE ACC_PARTNER_REGION = 'NAMR'
    AND TRUNC(DW_UPDATE_DT) >= TO_DATE('{oracle_date_format}', 'YYYY
-MM-DD')
    """
    df = spark.read
        .format("jdbc")
        .options(**ORACLE_SOURCE_CONFIG)
        .option("query", query)
        .load()
    return df
```

Step 4: Transforming the Extracted Data

The `transform\_data` function applies transformations to the extracted data. In this case, it primarily involves renaming fields.

```
def transform_data(df: DataFrame) -> DataFrame:
    transformed_df = df
        .withColumnRenamed("ACC_ACCOUNT_NUMBER", "ACC_ACCOUNT_DECIMAL")
        .withColumnRenamed("ACC_PROSPECT_NUMBER", "ACC_PROSPECT_DECIMAL")
    return transformed_df
```

Step 5: Loading Data to the Target Oracle Table

The `load\_data` function loads the transformed data to the target Oracle table.

```
def load_data(df: DataFrame, target_config: Dict[str, Any]) -> None:
    df.write
        .format("jdbc")
        .options(**target_config)
        .mode("append")
        .save()
```

Step 6: Orchestrating the ETL Process

The `extract\_transform\_load` function orchestrates the ETL process by calling the `extract\_data`, `transform\_data`, and `load\_data` functions in sequence.

```
def extract_transform_load(spark: SparkSession, update_dt: str) -> None:
    source_df = extract_data(spark, update_dt)
    transformed_df = transform_data(source_df)
    load_data(transformed_df, ORACLE_TARGET_CONFIG)
```

Step 7: Creating a Spark Session and Parsing Command-Line Arguments

The script creates a Spark session using the `create\_spark\_session` function and parses command-line arguments using the `parse\_arguments` function.

```
def create_spark_session() -> SparkSession:
    return SparkSession.builder
        .appName("SFDC Account Integration")
        .config("spark.sql.legacy.timeParserPolicy", "LEGACY")
        .config("spark.sql.adaptive.enabled", "true")
        .config("spark.sql.shuffle.partitions", "200")
        .getOrCreate()

def parse_arguments():
    parser = argparse.ArgumentParser(description='SFDC Account ETL Process')
    parser.add_argument('--update_dt', required=True,
                        help='Update date in MM/DD/YYYY format')
    return parser.parse_args()
```

Step 8: Executing the ETL Process

The script executes the ETL process by calling the `extract\_transform\_load` function with the parsed command-line arguments.

```
if __name__ == "__main__":
    args = parse_arguments()
    spark = create_spark_session()
    extract_transform_load(spark, args.update_dt)
```